

Static import

It allow you to access the static members (variables (or) methods) of the class. directly without using the class name

Syntax

import static packageName.className.staticMember;

Access modifiers

Control the visibility (scope) of classes, methods & variables.

Types

Public
(everywhere)

Protected
(package +
subclasses)

default
(package
only)

Private
(class only)

Encapsulation

Process of binding data (variables) & methods together into a single unit. (class) restrict direct access to the data from outside

encapsulation = data binding + data protection

- ↳ Private variables (use) ↳ Provide get & set methods access them safely.
- ↳ Prevents unauthorized access.

Get & set

In encapsulation, the data members (variables) are private, so they can't be accessed directly from the outside of the class.

Getter methods → To read (get) data

Getter

(Private variable)
setter

Setter methods → To write (set) data. [get or return [set or modify the value]]

Syntax:

```
Class className {
```

```
    Private dataType variableName;
```

```
// Getter
```

```
    Public void (dataType) getVariableName () {  
        return variableName;  
    }
```

```
// setter
```

```
    Public void setVariableName  
        (dataType value) {
```

```
        variableName = value;  
    }
```

Inheritance in Encapsulation

Private variable in a parent class can't be accessed directly in a child class even if the child class uses extends.

(Uses protected if the subclass access is intended)

Advantage

Data hiding
Data security
Code reusability
Easy maintenance

Package → Keyword (package)

Static import → Import static

Access modifiers → Public, Private, default, Protected.

Encapsulation → Private, getters, setters.

Abstraction

Process of hiding implementation details & shows the essential features of an object. [focuses on what an object does, not how it does it].

↳ Hiding the complex internal code & expose the necessary methods to the user.

Syntax

```
abstract class ClassName {  
    abstract void methodName();  
    void normalMethod() {  
        // Code  
    }  
}
```

Syntax (Tagged array)

```
int [][] jagged = new int [3][ ];
```

Tagged array

Advantage.

Saves memory
Faster access
Better representation
Flexible structure.

Method overloading (Compile Time poly)

It has multiple methods with the same name but different parameters in the same class. (which method to call before the program runs.)

↳ Known as Compile-Time Polymorphism

One class → Same method → different inputs. (ex) add (10, 20)
add (10, 20, 30)

Syntax

```
class ClassName {  
    returnType methodName (Parameters 1) {  
        // Code  
    }  
    returnType methodName (Parameter 1, Parameter 2) {  
        // Code  
    }  
}
```

}

↳ Same method name

↳ Same class

↳ different type of parameters

Method overriding. (Runtime poly)

A child class provides its own implementation of a method that already exists in the parent class. method name & parameter must be the same.

↳ Known as Run-time Polymorphism

→ Parent class says one thing, child class changed it.

Animal → sound() → "Animal makes a sound"

Cat → sound() → "Meow Meow".

Syntax.

```
class ParentClass {  
    returnType method Name () {  
        //code  
    }  
}  
  
class Childclass extends Parentclass {  
    returnType method Name () {  
        //code  
    }  
}
```

// parent class had a method.

// child class defines the same method (same name & parameters)

// replace the parent method (child).

Overriding = Parent class + child class + same parameters.